

PART A – THEORY QUESTIONS

Q1)What is the Collection Framework?

Ans:-The Java Collection Framework (JCF) is a set of classes and interfaces in Java that helps us store and manipulate groups of objects efficiently.

-It provides:

- Ready-made data structures (List, Set, Map, etc.)
- Methods for adding, removing, searching, sorting
- Dynamic memory handling (no fixed size like arrays)
- It is present in java.util package.

-Example:

```
ArrayList<Integer> list = new ArrayList<>();
```

Q2)What are the interfaces of the Collection Framework?

Ans:-Main interfaces of Collection Framework:

1)Iterable

2)Collection

3)List

4)Set

5)Queue

Map (separate hierarchy)

1.List

-Allows duplicates

- Maintains insertion order
- Examples: ArrayList, LinkedList, Vector

2. Set

- Does not allow duplicates
- No guaranteed order (except LinkedHashSet)
- Examples: HashSet, LinkedHashSet, TreeSet

3. Queue

- Follows FIFO (First In First Out)
- Examples: PriorityQueue, LinkedList

4. Map

- Stores data in key-value pairs
- Keys must be unique
- Examples: HashMap, LinkedHashMap, TreeMap

Q3)What is the difference between Collection and Collections?

Ans:-Collection is an interface available in the java.util package. It is the root interface of the Collection Framework (except Map). It represents a group of objects as a single unit. The Collection interface provides basic methods such as add(), remove(), size(), clear(), and contains(). Interfaces like List, Set, and Queue extend the Collection interface.

On the other hand, Collections is a utility class present in the java.util package. It provides static methods to perform operations on collection objects such as sorting, searching, reversing, shuffling, and synchronizing. Some commonly used methods are Collections.sort(), Collections.reverse(), and Collections.shuffle().

Q4)Difference between List, Set, and Map.

Ans:-List, Set, and Map are important interfaces in the Java Collection Framework used to store data in different ways.

1.List:-

- A List is an ordered collection that allows duplicate elements. -It maintains the insertion order and allows access to elements using an index.
- Classes like ArrayList, LinkedList, and Vector implement the List interface.
- List is useful when we need to maintain order and allow duplicate values.

2.Set:-

- A Set is a collection that does not allow duplicate elements.
- It does not guarantee insertion order (except LinkedHashSet). -
- The main purpose of Set is to store unique elements.
- Examples include HashSet, LinkedHashSet, and TreeSet.

3.Map:-

- A Map is different from List and Set because it does not extend the Collection interface.
- It stores data in the form of key-value pairs.
- In Map, keys must be unique, but values can be duplicated.
- Examples include HashMap, LinkedHashMap, and TreeMap. -
- Map is mainly used when data needs to be stored and retrieved using a key.

Q5)What is ArrayList?

Ans:-ArrayList is a class that implements the List interface.

-Features:

-Allows duplicates

-Maintains insertion order

-Dynamic size (can grow automatically)

-Allows random access using index

-Not synchronized

-Example:

```
import java.util.*;

public class Test {

    public static void main(String[] args) {

        ArrayList<String> list = new ArrayList<>();

        list.add("Java");

        list.add("Python");

        list.add("Java");

        System.out.println(list);

    }

}
```

PART B – PROGRAMMING QUESTIONS

Q1)Write a program to count occurrences of an element in an array.

Ans:-

```
import java.util.Scanner;

public class CountOccurrences {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter size of array:");

        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.println("Enter " + n + " elements:");

        for(int i = 0; i < n; i++) {

            arr[i] = sc.nextInt();

        }

        System.out.println("Enter element to count:");

        int key = sc.nextInt();

        int count = 0;

        for(int i = 0; i < n; i++) {

            if(arr[i] == key) {

                count++;

            }

        }

        System.out.println("Occurrences of " + key + " = " + count);

    }

}
```

Q2)Write a program to separate even and odd numbers from an array.

Ans:-

```
import java.util.Scanner;

public class SeparateEvenOdd {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter size of array:");

        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.println("Enter " + n + " elements:");

        for(int i = 0; i < n; i++) {

            arr[i] = sc.nextInt();

        }

        System.out.println("Even numbers are:");

        for(int i = 0; i < n; i++) {

            if(arr[i] % 2 == 0) {

                System.out.print(arr[i] + " ");

            }

        }

        System.out.println("\nOdd numbers are:");

        for(int i = 0; i < n; i++) {

            if(arr[i] % 2 != 0) {

                System.out.print(arr[i] + " ");

            }

        }

    }

}
```

```
    }  
  }  
}
```

Q3)Write a program to find intersection of two arrays.

Ans:-import java.util.Scanner;

```
    public class IntersectionArray {  
        public static void main(String[] args) {  
  
            Scanner sc = new Scanner(System.in);  
            System.out.println("Enter size of first array:");  
            int n1 = sc.nextInt();  
            int[] arr1 = new int[n1];  
  
            System.out.println("Enter elements of first array:");  
            for(int i = 0; i < n1; i++) {  
                arr1[i] = sc.nextInt();  
            }  
            System.out.println("Enter size of second array:");  
            int n2 = sc.nextInt();  
            int[] arr2 = new int[n2];  
  
            System.out.println("Enter elements of second array:");  
            for(int i = 0; i < n2; i++) {  
                arr2[i] = sc.nextInt();  
            }  
            System.out.println("Intersection elements are:");
```

```

for(int i = 0; i < n1; i++) {
    for(int j = 0; j < n2; j++) {
        if(arr1[i] == arr2[j]) {
            System.out.print(arr1[i] + " ");
            break; // avoid duplicate printing
        }
    }
}
}
}
}

```

Q4)Write a program to find union of two arrays.

Ans:-

```

import java.util.Scanner;

public class UnionArray2 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter size of first array:");

        int n1 = sc.nextInt();

        int[] arr1 = new int[n1];

        System.out.println("Enter elements of first array:");

        for(int i = 0; i < n1; i++) {

            arr1[i] = sc.nextInt();

        }
    }
}

```



```
System.out.println("Enter size of second array:");

int n2 = sc.nextInt();

int[] arr2 = new int[n2];

System.out.println("Enter elements of second array:");

for(int i = 0; i < n2; i++) {
    arr2[i] = sc.nextInt();
}

System.out.println("Union elements are:");

for(int i = 0; i < n1; i++) {
    System.out.print(arr1[i] + " ");
}

for(int i = 0; i < n2; i++) {
    boolean found = false;

    for(int j = 0; j < n1; j++) {
        if(arr2[i] == arr1[j]) {
            found = true;
            break;
        }
    }

    if(!found) {
        System.out.print(arr2[i] + " ");
    }
}

}
```

Q5)Write a program to check if two arrays are equal.

Ans:-

```
import java.util.Scanner;

public class CheckArrayEqual {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter size of first array:");

        int n1 = sc.nextInt();

        int[] arr1 = new int[n1];

        System.out.println("Enter elements of first array:");

        for(int i = 0; i < n1; i++) {

            arr1[i] = sc.nextInt();

        }

        System.out.println("Enter size of second array:");

        int n2 = sc.nextInt();

        int[] arr2 = new int[n2];

        if(n1 != n2) {

            System.out.println("Arrays are not equal (different sizes).");

            return;

        }

        System.out.println("Enter elements of second array:");

        for(int i = 0; i < n2; i++) {

            arr2[i] = sc.nextInt();
```

```
}  
  
boolean isEqual = true;  
  
for(int i = 0; i < n1; i++) {  
    if(arr1[i] != arr2[i]) {  
        isEqual = false;  
        break;  
    }  
}  
  
if(isEqual) {  
    System.out.println("Arrays are equal.");  
} else {  
    System.out.println("Arrays are not equal.");  
}  
  
}  
  
}
```